



NET: Networked Edge Training for Spatial Graph Learning

Group 2: Joe Conroy and Noah Constable
CSCI 8715, Spring 2026

Motivation

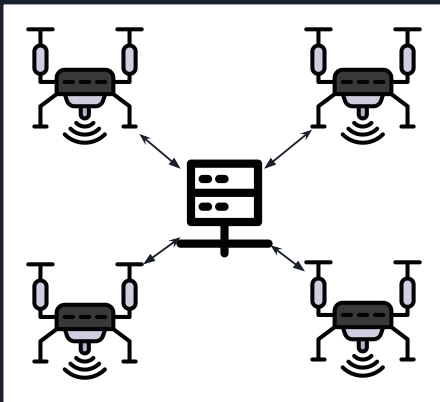
Our motivation is summed up by these questions:

- How is gossip learning different than federated average learning or centralized learning?
- Do gossip and federated average learning do better in spatial tasks?



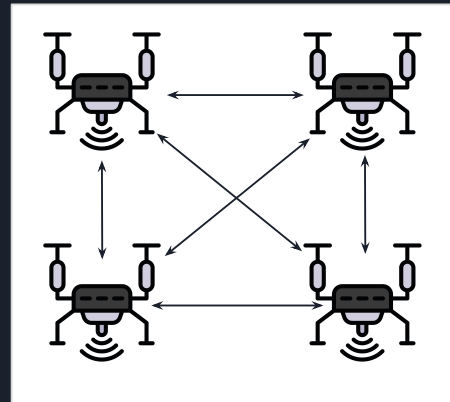
Centralized

VS.



Federated Averaging

VS.



Gossip

Problem Statement

- Task: Characterize gossip learning efficiency for spatial tasks vs federated and centralized learning
- Inputs
 - Spatial Environment Graph
 - Communication Graph
 - Local Observation Datasets
 - Task
- Outputs
 - Learned Model Parameters
 - Task Plan
 - Empirical Characterizations



Related Work & Limitations

Paper	Compare	Contrast
<u>Decentralized learning works: An empirical comparison of gossip learning and federated learning</u>	• Gossip vs Fed. Learning • Decentralized Optimization • Communication Efficiency • Real-World Device Traces	• Classification Task • Regression • Static Devices • Non-Autonomous Devices
<u>Peer-to-Peer Federated Learning on Graphs</u>	• Decentralized Framework • Communication Graph • Peer-to-Peer Updates	• Non-Spatial • Task Graph vs Communication Graph • No Analysis of Spatial Metrics
<u>A Graph Neural Network Based Decentralized Learning Scheme</u>	• GNNs • Decentralized • Graph Data	• No Path Planning • Communication Constraints • No Comparison of Gossip vs Fed.

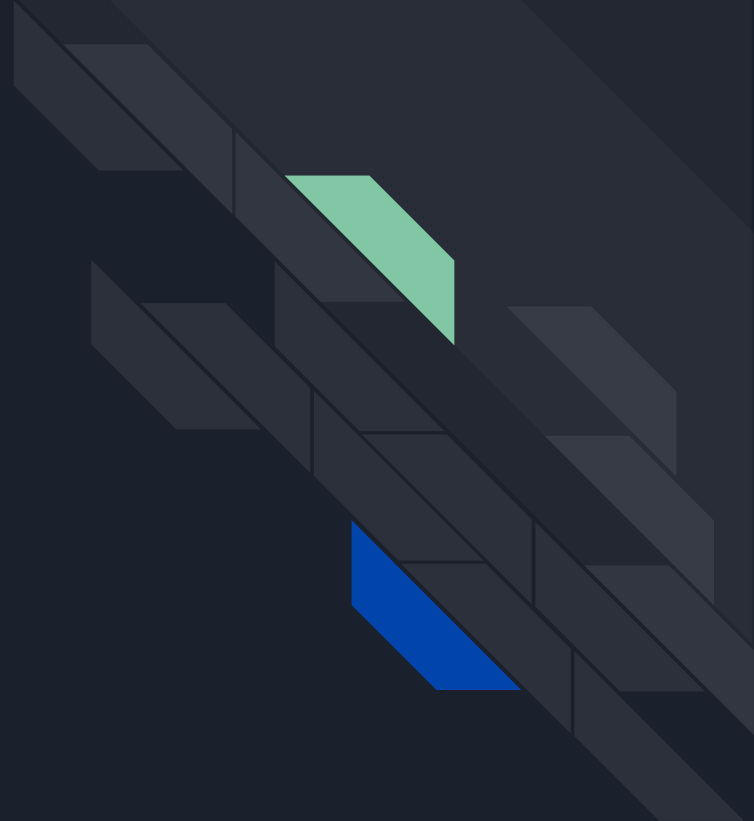


Approach

General flow:

- Load Data
- Construct Graph + Partitions
- Create Comm. Channels + Simulator
- Train:
 - Centralized Training
 - full-batch supervised learning on complete graph
 - Federated Average Training
 - subgraph learning with constraints, averaged on comm. rounds against central agent's model
 - Gossip Training
 - similar to fed. avg., but averaged against other nodes

Challenges and Resolutions



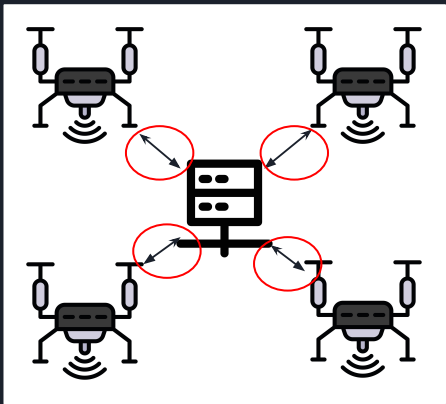
Challenge - Modeling Communication

Create comm. channels:

- Communication Channels to be used by the federated average + gossip models

- Parameters:

- `comm_every`: steps (rounds) between communication events
- `dropout_p`: per-drone communication dropout probability (per round)
- `baseline_p`: expected baseline dropout rate
 - rounds with dropout rates higher than this are marked as 'blackout' rounds
- `seed`: RNG seed for reproducibility



Challenge - Modeling Connectivity

1. Intermittent Connectivity

1.1. Peer-to-peer communication may be delayed or dropped

```
113
114 class CommunicationChannel:
115     """Model communication schedule and per-round dropout for K drones.
116
117     Parameters
118     -----
119     comm_every : int      - steps between communication events
120     dropout_p   : float   - per-drone independent dropout probability each round
121     baseline_p  : float   - expected baseline dropout rate; rounds exceeding this
122                           are flagged as blackout by the logger.
123                           Defaults to ``dropout_p``.
124     seed       : int      - RNG seed for reproducibility (``None`` = random)
125
126     Attributes
127     -----
128     logger : ProtocolInterruptionLogger
129             Attached automatically. Accessible for post-experiment analysis and
130             JSON export via ``channel.logger.save(path)``.
131     """
```

```
247
248 print("[3/6] Constructing communication channels and simulator...")
249 gossip_channel = CommunicationChannel(
250     comm_every=args.comm_every,
251     dropout_p=args.dropout_p,
252     baseline_p=args.baseline_p,
253     seed=args.seed,
254 )
255 fedavg_channel = CommunicationChannel(
256     comm_every=args.comm_every,
257     dropout_p=args.dropout_p,
258     baseline_p=args.baseline_p,
259     seed=args.seed,
260 )
```

Modeling Connectivity Cont.

```
378
379     if channel.is_comm_round(epoch):
380         if use_simulator:
381             participant_ids = channel.sample_participants(
382                 simulator.drones_within_radius(
383                     simulator.base_station_node(),
384                     simulator_comm_radius,
385                     drone_ids=list(range(len(nodes))),
386                 )
387             )
388         else:
389             participant_ids = channel.sample_participants(list(range(len(nodes))))
390             updates = [nodes[i].send_local_update() for i in participant_ids]
391
392         if updates:
393             central.aggregate_updates(updates)
394             central.broadcast_params()
```

Challenge - Modeling Resource Limitations

2. Limited Computation Capacity

2.1. Cannot train large models or run many local epochs

```
91 parser.add_argument(  
92     "--num-threads",  
93     type=int,  
94     default=4,  
95     help="CPU thread cap used during local drone training.",  
96 )  
97 parser.add_argument(  
98     "--time-budget",  
99     type=int,  
100     default=None,  
101     help="Optional per-drone local-training budget in milliseconds.",  
102 )
```

```
192 model.train()  
193  
194 _prev_threads = torch.get_num_threads()  
195 torch.set_num_threads(num_threads)  
196 _t0 = time.perf_counter()  
197  
198 losses: List[float] = []  
199  
200 for _ in range(epochs):  
201     optimizer.zero_grad()  
202     out = model(sub_x, sub_edge_index)  
203     loss = F.mse_loss(out, sub_y)  
204     loss.backward()  
205     optimizer.step()  
206     losses.append(loss.item())  
207     if (  
208         ..... time_budget_ms is not None  
209         ..... and (time.perf_counter() - _t0) * 1000 >= time_budget_ms  
210         ..... ):  
211         break  
212  
213 torch.set_num_threads(_prev_threads)  
214
```

Centralized Training

Centralized training:

- Trains full model on full graph over N epochs
 - Parameters:
 - `data`: PyG data (full graph)
 - `model`: the model we are training
 - `epochs`: the number of epochs
 - `lr`: learning rate
- PyG makes this pretty straightforward!

```
83 def train_centralized(  
84     data: Data,  
85     model: WetlandGCN,  
86     epochs: int,  
87     lr: float = 1e-3,  
88 ) -> list[float]:  
89     """Train *model* on the full graph for *epochs* steps.  
90  
91     Parameters  
92     -----  
93     data    : PyG Data with fields x (N,1) and y (N,1)  
94     model   : WetlandGCN instance; updated in-place  
95     epochs  : number of gradient steps  
96     lr      : Adam learning rate  
97  
98     Returns  
99     -----  
100    losses : list of per-epoch MSE loss values (length == epochs)  
101    """  
102    optimizer = torch.optim.Adam(model.parameters(), lr=lr)  
103    model.train()  
104  
105    losses: list[float] = []  
106    for _ in range(epochs):  
107        optimizer.zero_grad()  
108        out = model(data.x, data.edge_index)  
109        loss = F.mse_loss(out, data.y)  
110        loss.backward()  
111        optimizer.step()  
112        losses.append(loss.item())  
113  
114    return losses  
115
```

Federated Average Training

More intense than centralized!

- Notable parameters:
 - `partitions`: partitions of the full graph
 - `channel`: communications channel for this agent
 - `num-threads`: number of CPU threads that Torch will be given
 - `time_budget_ms`: time limit for each epoch
 - `simulator`: Spatial grid simulator to track positions and movements
 - `snaphshot_x`: debug image snapshot configurations

```
288 def train_fedavg(  
289     data: Data,  
290     partitions: list[np.ndarray],  
291     channel: CommunicationChannel,  
292     epochs: int,  
293     local_steps: int,  
294     lr: float = 1e-3,  
295     num_threads: int = 4,  
296     time_budget_ms: int | None = None,  
297     initial_state_dict: Dict[str, torch.Tensor] | None = None,  
298     hidden_channels: int = 64,  
299     simulator: SpatialGridSimulator | None = None,  
300     simulator_view_radius: int = 2,  
301     simulator_comm_radius: int = 2,  
302     move_drones: bool = False,  
303     snapshot_every: int = 0,  
304     snapshot_output_dir: str | Path | None = None,  
305     snapshot_method_name: str = "FedAvg",  
306 ) -> tuple[list[float], WetlandGCN]:  
307     """
```

Fed. Avg. Training Cont.

1) Create Central And Node Agents:

```
317     if initial_state_dict is None:
318         template_model = WetlandGCN(
319             in_channels=int(data.x.shape[1]), hidden_channels=hidden_channels
320         )
321         initial_state_dict = _clone_state_dict(template_model.state_dict())
322
323     central = CentralAgent()
324     nodes = [NodeAgent(node_id=f"node_{i}") for i in range(len(partitions))]
325     for node in nodes:
326         central.register_node(node)
327
328     central.global_params = _clone_state_dict(initial_state_dict)
329     central.broadcast_params()
```

Fed. Avg. Training Cont.

2) Configure simulator and partitions

```
329     central.broadcast_params()
330     use_simulator = simulator is not None
331     if snapshot_every > 0 and not use_simulator:
332         raise ValueError("Snapshot export requires a simulator-backed FedAvg run.")
333     if snapshot_every > 0 and snapshot_output_dir is None:
334         raise ValueError("snapshot_output_dir is required when snapshot_every > 0.")
335     local_partitions = None
336     if not use_simulator:
337         local_partitions = [build_local_subgraph(data, partition) for partition in partitions]
338
339     global_loss_curve: list[float] = []
340
341     if use_simulator and _should_save_snapshot(0, epochs, snapshot_every):
342         simulator.save_snapshot(
343             epoch=0,
344             output_dir=snapshot_output_dir,
345             method_name=snapshot_method_name,
346             node_values=_predict_state_dict(data, central.global_params, hidden_channels),
347             metric_label="MSE",
348             metric_value=_evaluate_state_dict(data, central.global_params, hidden_channels),
349         )
```

Fed. Avg. Training Cont.

3.1) Enter epoch loop -> train local subgraphs

```
357     # Each node trains locally on its partition
358     for i, node in enumerate(nodes):
359         if use_simulator:
360             node.current_loc = drone_positions.get(i)
361             local_training_view = simulator.get_local_view(
362                 i, radius=simulator_view_radius
363             )
364         else:
365             local_training_view = local_partitions[i]
366
367         node.train_local(
368             local_training_view,
369             location=None if use_simulator else node.current_loc,
370             radius=simulator_view_radius,
371             epochs=local_steps,
372             lr=lr,
373             num_threads=num_threads,
374             time_budget_ms=time_budget_ms,
375             use_full_graph=True,
376             hidden_channels=hidden_channels,
377         )
378
```

Fed. Avg. Training Cont.

3.2) Sample participant drones on comm. round, update global average

```
378
379     if channel.is_comm_round(epoch):
380         if use_simulator:
381             participant_ids = channel.sample_participants(
382                 simulator.drones_within_radius(
383                     simulator.base_station_node(),
384                     simulator_comm_radius,
385                     drone_ids=list(range(len(nodes))),
386                 )
387             )
388         else:
389             participant_ids = channel.sample_participants(list(range(len(nodes))))
390             updates = [nodes[i].send_local_update() for i in participant_ids]
391
392             if updates:
393                 central.aggregate_updates(updates)
394                 central.broadcast_params()
395
396     avg_state = _average_state_dicts([node.model_params for node in nodes])
397     global_loss_curve.append(_evaluate_state_dict(data, avg_state, hidden_channels))
398
```

Fed. Avg. Training Cont.

3.3) Increment epoch counter, save snapshot of simulator

```
398
399     completed_rounds = epoch + 1
400     if use_simulator and _should_save_snapshot(
401         completed_rounds, epochs, snapshot_every
402     ):
403         simulator.save_snapshot(
404             epoch=completed_rounds,
405             output_dir=snapshot_output_dir,
406             method_name=snapshot_method_name,
407             node_values=_predict_state_dict(data, avg_state, hidden_channels),
408             metric_label="MSE",
409             metric_value=global_loss_curve[-1],
410         )
411
```

4) Compute and return final model from averages

```
412     final_model = WetlandGCN(
413         in_channels=int(data.x.shape[1]), hidden_channels=hidden_channels
414     )
415     final_model.load_state_dict(avg_state)
416
417     return global_loss_curve, final_model
418
```

Gossip Training

Very similar to Fed.Avg.

- `partitions`: partitions of the full graph
- `channel`: communications channel for this agent
- `num-threads`: number of CPU threads that Torch will be given
- `time_budget_ms`: time limit for each epoch
- `simulator`: Spatial grid simulator to track positions and movements
- `snapshot_x`: debug image snapshot configurations

```
121 def train_gossip(  
122     data: Data,  
123     partitions: list[np.ndarray],  
124     channel: CommunicationChannel,  
125     epochs: int,  
126     local_steps: int,  
127     lr: float = 1e-3,  
128     num_threads: int = 4,  
129     time_budget_ms: int | None = None,  
130     initial_state_dict: dict[str, torch.Tensor] | None = None,  
131     hidden_channels: int = 64,  
132     simulator: SpatialGridSimulator | None = None,  
133     simulator_view_radius: int = 2,  
134     simulator_comm_radius: int = 2,  
135     move_drones: bool = False,  
136     snapshot_every: int = 0,  
137     snapshot_output_dir: str | Path | None = None,  
138     snapshot_method_name: str = "Gossip",  
139 ) -> tuple[list[float], list[WetlandGCN]]:  
140     """Fully decentralised gossip training.  
141
```

Gossip Training Cont.

1) Create 'drones' from partitions, create and assign models + model params

```
194
195     K = len(partitions)
196     drone_ids = list(range(K))
197
198     if initial_state_dict is None:
199         template_model = WetlandGCN(
200             in_channels=int(data.x.shape[1]), hidden_channels=hidden_channels
201         )
202         initial_state_dict = _clone_state_dict(template_model.state_dict())
203
204     drone_models: list[WetlandGCN] = [
205         WetlandGCN(in_channels=int(data.x.shape[1]), hidden_channels=hidden_channels)
206         for _ in range(K)
207     ]
208     for model in drone_models:
209         model.load_state_dict(_clone_state_dict(initial_state_dict))
210     optimizers = [torch.optim.Adam(m.parameters(), lr=lr) for m in drone_models]
```

Gossip Training Cont.

2) Configure simulator

```
211     use_simulator = simulator is not None
212     if snapshot_every > 0 and not use_simulator:
213         raise ValueError("Snapshot export requires a simulator-backed gossip run.")
214     if snapshot_every > 0 and snapshot_output_dir is None:
215         raise ValueError("snapshot_output_dir is required when snapshot_every > 0.")
216     local_subgraphs = None
217     if not use_simulator:
218         local_subgraphs = [build_local_subgraph(data, idx) for idx in partitions]
219
220     losses: list[float] = []
221
222     if use_simulator and _should_save_snapshot(0, epochs, snapshot_every):
223         initial_avg_state = _average_state_dicts(
224             [model.state_dict() for model in drone_models]
225         )
226         simulator.save_snapshot(
227             epoch=0,
228             output_dir=snapshot_output_dir,
229             method_name=snapshot_method_name,
230             node_values=_predict_state_dict(data, initial_avg_state, hidden_channels),
231             metric_label="MSE",
232             metric_value=_evaluate_state_dict(data, initial_avg_state, hidden_channels),
233         )
234
```

Gossip Training Cont.

3.1) Enter epoch loop -> get subgraphs from local positions

```
235     for comm_round in range(epochs):
236         if use_simulator and move_drones:
237             simulator.step_drones()
238
239         round_subgraphs = (
240             [
241                 simulator.get_local_view(drone_id, radius=simulator_view_radius)
242                 for drone_id in drone_ids
243             ]
244             if use_simulator
245             else local_subgraphs
246         )
247
```

Gossip Training Cont.

3.2) Locally train V steps

```
248     # 1. Local training - apply per-drone compute constraints
249     for model, opt, sub in zip(drone_models, optimizers, round_subgraphs):
250         model.train()
251         _prev_threads = torch.get_num_threads()
252         torch.set_num_threads(num_threads)
253         _t0 = time.perf_counter()
254         for _ in range(local_steps):
255             opt.zero_grad()
256             out = model(sub.x, sub.edge_index)
257             loss = F.mse_loss(out, sub.y)
258             loss.backward()
259             opt.step()
260             if (
261                 time_budget_ms is not None
262                 and (time.perf_counter() - _t0) * 1000 >= time_budget_ms
263             ):
264                 break
265         torch.set_num_threads(_prev_threads)
266
```

Gossip Training Cont.

3.3) Exchange models ("gossip") with neighbor drones

```
266 |
267 |     # 2. Gossip exchange – pairs determined by the channel
268 |     if channel.is_comm_round(comm_round):
269 |         if use_simulator:
270 |             allowed_pairs = simulator.proximity_pairs(
271 |                 simulator_comm_radius, drone_ids=drone_ids
272 |             )
273 |             pairs = channel.gossip_pairs(drone_ids, allowed_pairs=allowed_pairs)
274 |         else:
275 |             pairs = channel.gossip_pairs(drone_ids)
276 |         for i, j in pairs:
277 |             sd_i = _clone_state_dict(drone_models[i].state_dict())
278 |             sd_j = _clone_state_dict(drone_models[j].state_dict())
279 |             avg_sd = {k: (sd_i[k] + sd_j[k]) / 2.0 for k in sd_i}
280 |             drone_models[i].load_state_dict(avg_sd)
281 |             drone_models[j].load_state_dict(copy.deepcopy(avg_sd))
282 |             # Reset optimizers so stale moment estimates don't pollute new weights
283 |             optimizers[i] = torch.optim.Adam(drone_models[i].parameters(), lr=lr)
284 |             optimizers[j] = torch.optim.Adam(drone_models[j].parameters(), lr=lr)
```

Gossip Training Cont.

4) Exit epoch loop -> compute global average -> snapshot simulator state

```
286     avg_global = _average_state_dicts(  
287         [model.state_dict() for model in drone_models]  
288     )  
289     losses.append(_evaluate_state_dict(data, avg_global, hidden_channels))  
290  
291     completed_rounds = comm_round + 1  
292     if use_simulator and _should_save_snapshot(  
293         completed_rounds, epochs, snapshot_every  
294     ):  
295         simulator.save_snapshot(  
296             epoch=completed_rounds,  
297             output_dir=snapshot_output_dir,  
298             method_name=snapshot_method_name,  
299             node_values=_predict_state_dict(data, avg_global, hidden_channels),  
300             metric_label="MSE",  
301             metric_value=losses[-1],  
302         )  
303  
304     return losses, drone_models  
305
```



Validation

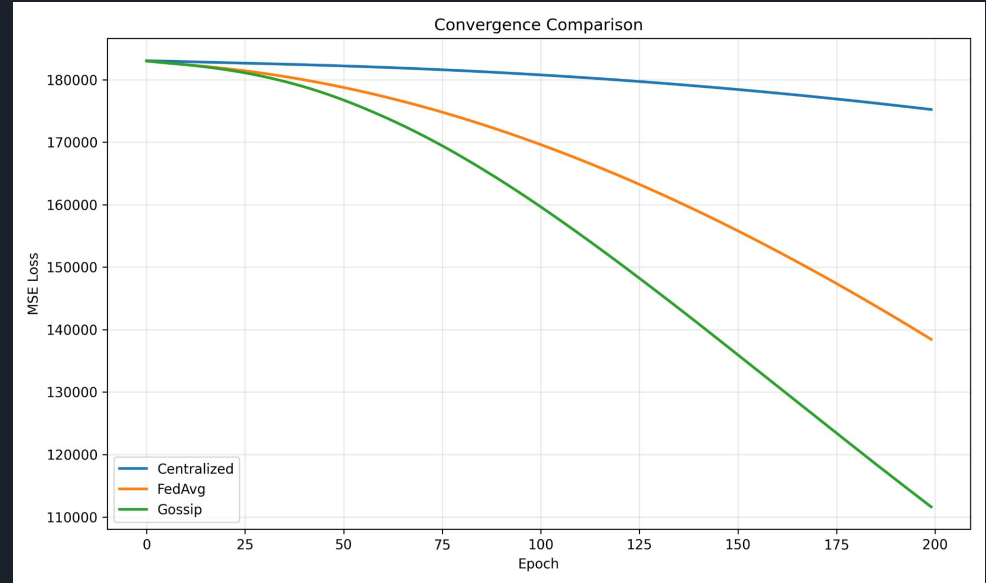
Fully Simulated

Experimental Design:

- Training Strategies
 - Centralized GNN
 - Federated-Average Learning
 - Gossip-Based Decentralized Learning
- Connectivity Conditions
 - Frequent synchronization
 - Sparse synchronization
 - Random dropouts
 - Time-varying communication graph
- Partition Schemes
 - Clean spatial partitions
 - Overlapping regions
 - Random assignment (control)
- Computation Constraints
 - Limited local epochs
 - Smaller hidden dimensions
 - Reduced precision (float32 vs float16)
 - Gradient sparsification

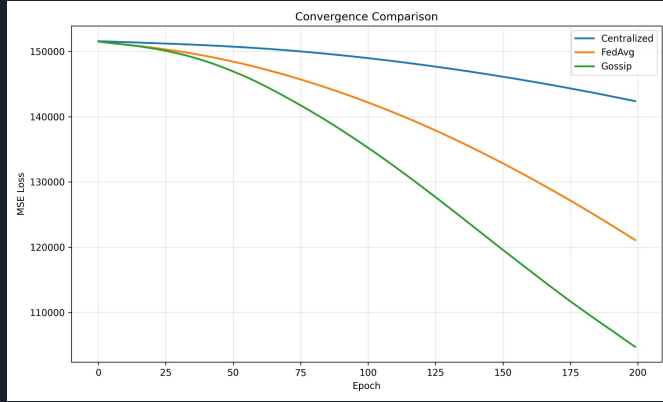
Conclusions (Contributions)

- Gossip models consistently outperformed baseline and federated method
- Decreased communication only hurt federated and centralized performance
- Increasing the dropout rate always worsened each model's overall performance

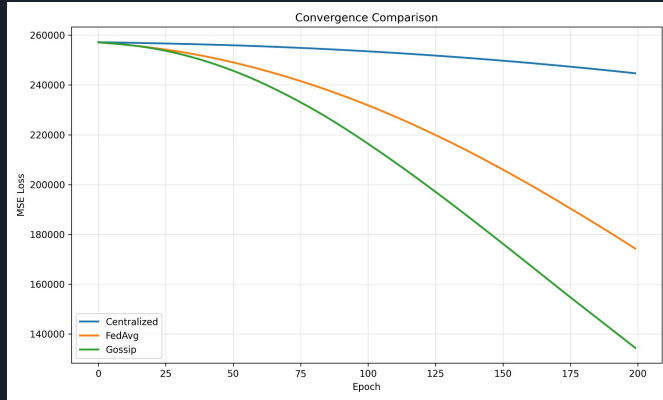


Thread and Time Budget Relationship

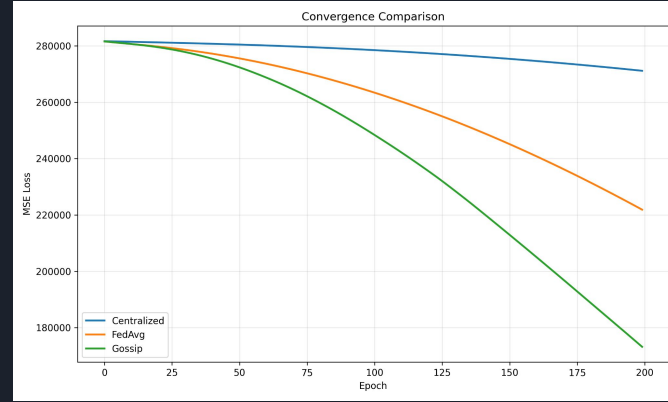
2 threads,
1000 time
budget



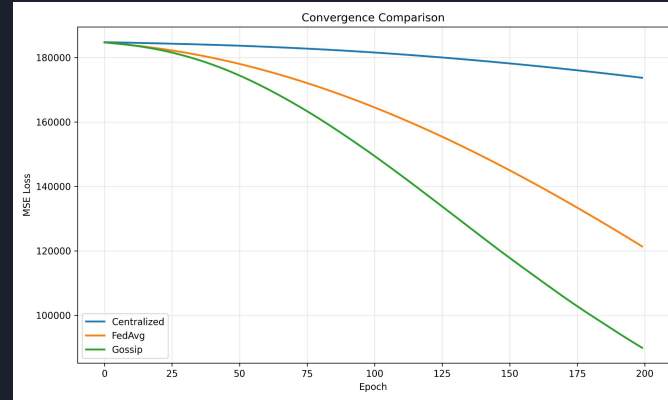
4 threads,
1000 time
budget



2 threads,
2000 time
budget



4 threads,
2000 time
budget





Shortcomings

- Greedy tests never converged
 - No path data logged to study behavior
- Communication interruptions mislogged as “blackout” drones
- Dropout rates had little variance
- Visualizations isolated to each run
 - Shared labels without specification differences



Future Work

- Rework map sampling
 - Use smaller chunks of the source data
 - Cache chunked data (performance improvement)
- Rework objective
 - Shortest path regression analysis -> continuous risk regression
- Rework graph partitioning
 - Currently use column-wise partitioning
- Rework movement
 - Currently relies on the partitioning
 - Free movement and logic to guide it
- Post-Training Trials
 - Test against unseen data
 - Add more analysis and visualizations
- Characterize Downstream Tasks



Resources

Data: National Wetland Inventory for Minnesota [DNR+St. Mary's University]
[\[link\]](#)

Code: Github [\[link\]](#)

Questions?

